CCDSALG Term 3, AY 2025 – 2026

MCO1 DOCUMENTATION – **Convex Hull (Application of Stack Data Structure & Sorting Algorithms)**

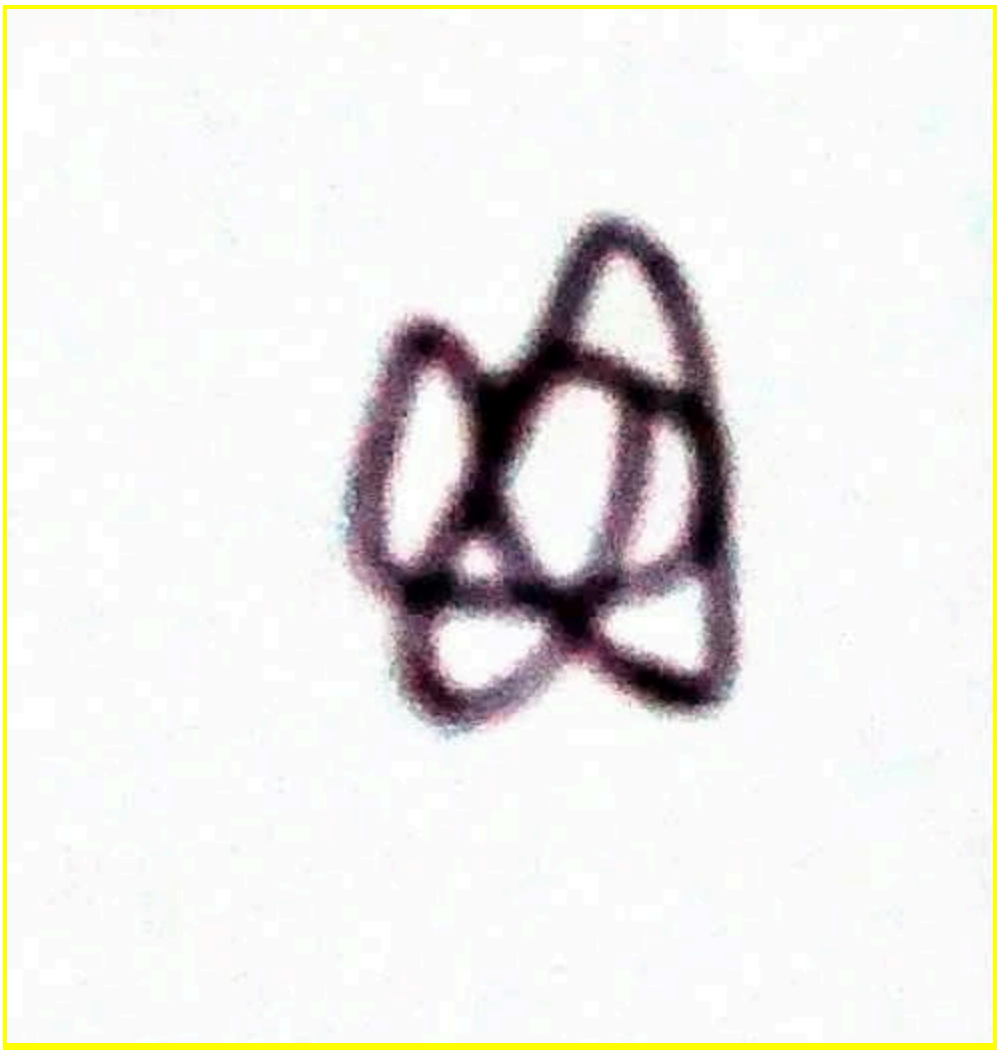
I. DECLARATION OF INTELLECTUAL HONESTY & ETHICAL USE OF GENERATIVE AI

1. *We declare that the project we are submitting is the product of our own work augmented with responsible and ethical use of generative AI as encoded in Table 1 and Table 2 below.*
2. *We declare that all AI-assisted contributions have been critically examined by our group. We acknowledge the limitations of Generative AI systems, including possible errors or biases, and accept full responsibility and accountability for all contents in all our project deliverables.*
3. *We also declare that each member COOPERATED and contributed EQUALLY to the project as detailed in Table 3 below.*
4. *No part of our work was shared with another person outside of our group.*

// Instruction #1: Encode your names, and affix your e-signature to attest to the declaration above.

ARNEDO, RAPHAEL ADAM C. S09

ACIERTO, JOAQUIN ELIJAH S04



// Instruction #2: Fill-up the column 2 of Table 1 below.

For each of the following tasks, identify the degree of AI use among the following:

- **NONE** - No AI was used for this task
- **SCAFFOLDING** - AI was used to build the foundation for this task, e.g., brainstorming, solution ideation
- **REFINEMENT** - AI was used to review, provide feedback, and revise outputs for this task
- **GENERATION** - AI was used to generate portions of the output for this task

Table 1. Responsible and Ethical Use of AI Details

TASKS	DEGREE OF AI USE & BRIEF EXPLANATION
T1: Understand the project specifications (problem) & assign responsibilities.	*NONE - we used other materials such as youtube to explain the concept of convex hull and the different types of algorithms such as graham scan and jarvis algorithm. We also used visualgo to visualize how it works*
T2: Design, implement and test your stack data structure module.	*NONE - based the code from youtube videos and from the definition on the slides*

T3: Implement and test your TWO sorting algorithms module.	*SCAFFOLDING - we used youtube materials to get a gist on how to compare points and on how to find the polar angles but we used AI to get an idea on how to find the lowest point*
T4: Implement two versions of Graham's Scan algorithm.	*SCAFFOLDING - used youtube materials to understand the implementation of stack operations and further used AI to get an even better understanding*
T5: Implement two driver programs to test the two Graham's Scan algorithm versions.	*NONE - we changed a couple of data type parameters of the cv function and the stack functions to accommodate the stack Point data type used to display the slow/fast algorithms*
T6: Black box test your Graham's Scan Algorithm implementation.	*NONE - we just made another c file that creates txt files that contains the size and 2^n amount of points and calculated the average ourselves.*
Task 7. Document your project.	*We wrote our documentation and made the graph ourselvesI.*

// Instruction #3: Fill up Table 2. List explicitly ALL AI tools that you employed, and justify their purpose/role in your project.

Table 2. List of AI Tools Used

AI TOOL USED (embed the link)	JUSTIFICATION/PURPOSE/ROLE
https://gemini.google.com/app	We used this to get ideas.

//… add more lines as you deem necessary

// Instruction #4: Fill up columns 1 and 2 of Table 3.

Table 3. Individual Contributions to the Project

NAME	CONTRIBUTION DETAILS
ARNEDO, RAPHAEL ADAM C.	Encode details of your contributions for each task. Indicate your role, for example, were you the project manager, were you the (lead) programmer for a certain module, were you the (black box) tester for a certain module, which part of the documentation were you responsible for? etc… Encode NONE if you did not contribute to a particular task. T1: Found youtube materials and attempted to understand the Rosetta code T2: Coded the stack functions and helper functions and stack data structure based from the Rosetta code and other youtube materials T3: Tested the insertion algo code and redid the Polar angles function T4: Made the stack operations work in a manner that it follows graham scan T5: Made the fast and slow graham scan algo T6: None T7:Helped in making the documentation

ACIERTO, JOAQUIN ELIJAH T.	T1: Read the pdf carefully, planned the responsibilities out. T2: Tested the Stack Ops multiple times, recommended revisions and changes. Fixed bugs. Provided the comments and documentation within the code. T3: Created the sort functions, the helper functions and test_sort.c file. Took inspiration from math topics with the use of distance formula and point-to-angle conversion. Provided the comments and documentation within the code. T4: Provided the comments and documentation within the code. Tested the code and provided bug fixing, cleaned up the code and removed any compiler errors or unused code. T5: Provided the comments and documentation within the code. Tested the code and provided bug fixing, cleaned up the code and removed any compiler errors or unused code. T6: Made the program that randomly generated 10 test case files. Did the black box testing. Provided averages for the outputs. T7: Helped in some parts of the documentation. Mostly did the graph and performance check table
LASTNAME3, FIRSTNAME3	T1: T2: T3: T4: T5: T6: T7:

II. SUBMISSION CHECKLIST

// Instruction #5: Fill up Column 3 of Table 4 below.

Put a check mark as specified in the 3rd column of the table below. Please make sure that you use the same file names and that you encoded the appropriate file contents. You may create/add other source files (for example, for reading and writing data).

Table 4. File Submission Checklist

FILE	DESCRIPTION	Put a check mark ✔ below to indicate that you submitted a required file
`stack.h`	stack data structure header file	✔
`stack.c`	stack data structure C source file	✔
`test_stack.c`	driver program for the stack module	✔
`sort.h`	“slow” and “fast” sorting algo header file	✔
`sort.c`	“slow” and “fast” sorting algo C source file	✔
`test_sort.c`	driver program for the sorting module	✔
`graham_slow.c`	Graham’s Scan algorithm slow version (using the “slow” sorting algorithm)	✔
`graham_fast.c`	Graham’s Scan algorithm fast version (using the “fast” sorting algorithm)	✔
`test_slow.c`	drive program for the “slow” version	✔

test_fast.c	drive program for the for the "fast" version	✔
INPUT1.TXTto **INPUT10.TXT**	10 sample input files (with increasing values of *n*)	✔
OUTPUT1.TXT to **OUTPUT10.TXT**	10 sample corresponding output files	✔
GROUPNUMBER.PDF	the PDF file of this document	✔

III. INSTRUCTIONS ON HOW TO COMPILE & RUN THE PROGRAMS

// Instruction #6: Indicate how to compile your source files, and how to RUN your exe files from the COMMAND LINE.

Examples are shown below highlighted in yellow. Replace them accordingly. Make sure that all your group members test what you typed below because I will follow them verbatim (copy/paste as is). I will initially test your solution using a sample input text file that you submitted. Thereafter, I will run it again using my own test data:

- How to compile from the command line
  C:\MCO> gcc -Wall test_slow.c -o slow.exe
  C:\MCO> gcc -Wall test_fast.c -o fast.exe

- How to run from command line
  C:\MCO>test_slow.exe
  C:\MCO>test_fast.exe

Next, answer the following questions:

a. Is there a compilation (syntax error) in your codes? (YES or NO). _NO_
   WARNING: the project will automatically be graded with a score of **0** if there is syntax error in any of the submitted source code files. Please make sure that your submission does not have a syntax error.

b. Is there any compilation warning in your codes? (YES or NO) **_NO_**
   WARNING: there will be a 1 point deduction for every unique compiler warning. Please make sure that your submission does not have a compiler warning.

IV. DISCLOSURE OF ERROR(S)

// Instruction #7: Disclose IN DETAIL what is/are NOT working correctly in your solution.

Please be honest about this. NON-DISCLOSURE will result in severe point deduction! Explain briefly the reason why your group was not able to make it work.

The following are NOT working (buggy):
a.
b.

We were not able to make them work because:
a.
b.

V. TEST RESULTS

// Instruction #8: Fill up Table 5 below.

Based on the exhaustive empirical testing that you did, fill-up the Comparison Table below that shows the performance between the "slow" version versus the "fast" version. Test for 10 different values of *n*, starting with *n = 2^6 = 64* points.

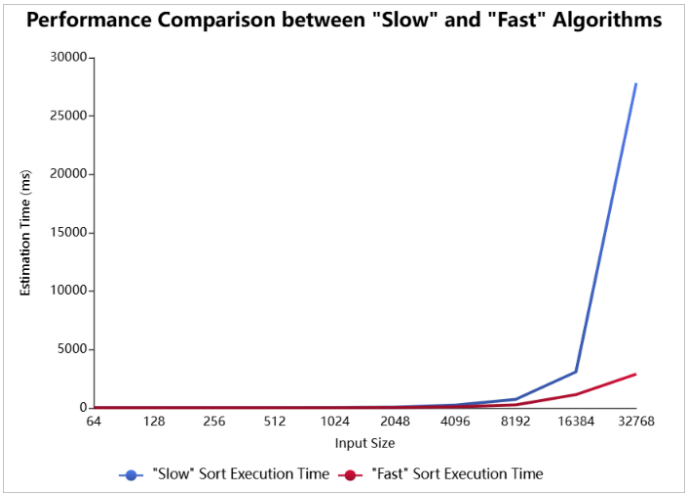
Table 5. Performance Comparison Between "Slow" and "Fast" Versions

Test Case #	n (input size)	"Slow" version execution time in *ms*	"Fast" version execution time in *ms*
1	*2^6 = 64*	0.56ms	0.67ms
2	*2^7*	0.71ms	0.68ms
3	*2^8*	1.06ms	1.03ms
4	*2^9*	2.88ms	1.82ms
5	*2^10*	21.91ms	8.79ms
6	*2^11*	59.18ms	23.56ms
7	*2^12*	248.91ms	90.15ms
8	*2^13*	729.06ms	266.30ms
9	*2^14*	3,091.42ms	1,140.42ms
10	*2^15 = 32768*	27,849.36ms	2896.73ms

NOTE: Make sure that you fill-up the table properly. It contributes 4 out of 15 points for the Documentation.

// Instruction #9: Provide a graph showing a visual comparison of the "slow" and "fast" versions.

Create a graph (for example using Excel) based on the Comparison Table that you filled-up above. The x-axis should be the values of *n* and the y axis should be the execution time in milliseconds (ms). **There should be two line graphs superimposed on the same xy axes, one for the "slow" and the other for the "fast".** DO NOT SUBMIT TWO separate graphs. Replace the image below with your own.



NOTE: Make sure that you provide a graph based on your comparison table data above. It contributes 4 out of 15 points for the Documentation.

VI. ANALYSIS OF RESULTS

// Instruction #10: Compare and analyze the growth rate behaviors of the "slow" and "fast" versions based on the Comparison Table and the graphs above.

Answer the following questions:

a. What do you think is the growth rate behavior of the "slow" version?

Insertion sort picks an element of the array and back tracks through all the elements with a lesser index to see if it fulfills a condition. Once the size becomes incredibly large such as 2^15, it would have to back track multiple times with that size.

b. What do you think is the growth rate behavior of the "fast" version?

Quick sort picks a pivot in which it checks those greater than it and less than it and partitions it. Sorting those individually in a recursive function. Quick sort's growth behavior seems to grow bigger the bigger the input size is

c. What do you think is/are the factor/s that make the "fast" version compute the results faster than the "slow" version?

Quick Sort works by dividing the array into groups, while Insertion works by comparing indexes individually. When the input size grows, quick sort is better in handling those because Insertion sort has to manually check each index while Quick sort is able to divide his responsibilities into large groups in which it sorts.

NOTE: Make sure that you provide cohesive answers to the three questions above. This part contributes 4 out of 15 points for the Documentation.

VII. SELF-EVALUATION

// Instruction #11: Fill-up Table 6 below.

Refer to the rubric in the project specs. It is suggested that you do an individual self-assessment first. Thereafter, compute the average evaluation for your group, and encode it below.

Table 6. Self-Evaluation

REQUIREMENT	AVE. OF SELF-ASSESSMENT
1. Stack	20 (max. 20 points)
2. Sorting algorithms	8 (max. 10 points)
3. Graham's Scan algorithm	35 (max. 40 points)
4. Project Documentation and Analysis	13 (max. 15 points)
5. Compliance with Instructions	5 (max. 5 points)

TOTAL SCORE _81_ over 90.

Note that the remaining 10 points (over 100) will come from the project demo.

NOTE: The evaluation that the instructor will give is not necessarily going to be the same as what you indicated above. The self-assessment is for your own reference only…

– The End –

サルバドル・フロランテ